AdaptConfig

AI-Powered Integration Configuration Platform
for Enterprise Lending

Team Nucleolus · IIT Patna

S Akash (Lead) · Swayam Jain · Yash Kamdar

FinSpark Hackathon · April 2026

https://github.com/Akasxh/adaptconfig

The Problem

- Credit Bureaus (CIBIL)
- Aadhaar eKYC
- GST Verification
- Payment Gateways
- Fraud Detection
- SMS / Email / AA

Pain Points

- **2–4 weeks** per integration
- Field mapping errors → data loss
- No pre-production testing
- 8 APIs = **16–32 weeks total**

Delay: 8 APIs × 2–4 weeks each → **4–8 months to launch**

Our Solution

Upload API Spec $\xrightarrow{\text{AI}}$ **Auto-Config** $\xrightarrow{\text{Test}}$ **Deploy**

From **weeks** to **minutes**. Zero manual field mapping.

Metric	Result	
Endpoints parsed	4	from CIBIL v2 spec
Fields extracted	28	with types & required flags
Auth schemes	2	OAuth2 + API Key
Parse confidence	95%	
Fields mapped	28/28	at 100% confidence
Simulation	8/8 PASSED	all steps green
Total time	$\sim$ 40s	end to end

Architecture

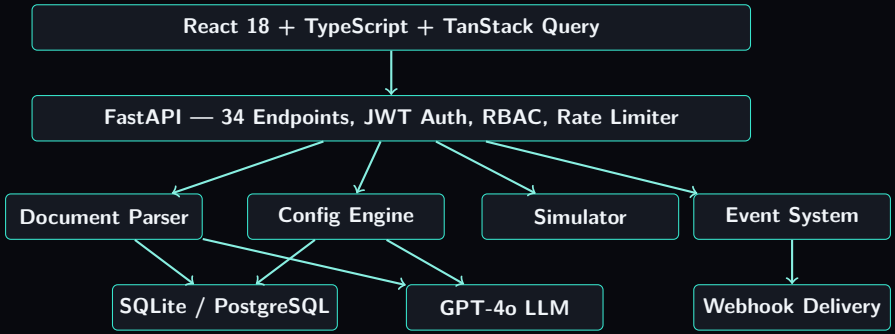
React Frontend $\xrightarrow{\text{JWT}}$ **FastAPI Backend (34 endpoints)** $\rightarrow$ **PostgreSQL**

Layer	Technologies
Frontend	React 18 · TypeScript · Tailwind CSS · Recharts
Backend	FastAPI · SQLAlchemy (async) · Pydantic v2
AI Engine	OpenAI GPT-4o (REST API)
Services	Document Parser · Config Engine · Simulator
Database	PostgreSQL (prod) · SQLite (dev)
Deploy	Railway · Docker · nginx · Alembic migrations
Auth	JWT · PBKDF2 · RBAC · per-user tenant isolation

Backend serves 34 API endpoints across 10 route modules.

AI pipeline: GPT-4o generates config $\rightarrow$ Rule engine validates with fuzzy matching.

Architecture Deep-Dive



Config Generation Pipeline

Hybrid AI + Rule Engine:

1. GPT-4o generates config from document + adapter schema
2. **Rule engine** augments with:
   - 100+ Indian fintech synonyms
   - Fuzzy string matching (rapidfuzz)
   - Jaccard token overlap
3. Rule-based confidence **overrides** LLM
4. Unmapped fields **backfilled**

Graceful degradation:

Path	When
LLM + Rules	API key present
Rule fallback	LLM fails
Pure rules	AI disabled

Field matching tiers:

1. Exact synonym → 1.0
2. Fuzzy match → 0.6–0.99
3. Token Jaccard → 0.6+

Rollback & Version Control

Configuration Lifecycle:

$$\text{Draft} \rightarrow \text{Configured} \rightarrow \text{Validating} \rightarrow \text{Testing} \rightarrow \text{Active}$$

Version history:

- Every change creates a snapshot
- Full state serialization
- One-click rollback to any version
- Pre-rollback snapshot for safety

Diff engine:

- Recursive dict/list comparison
- Breaking change detection
- Side-by-side version comparison
- Export as JSON or YAML

Audit trail:

- Immutable log of every action
- Actor, timestamp, resource details
- Per-user data isolation

Database Schema (11 Tables)

Table	Purpose	Key Relations
`users`	Auth, RBAC (admin/editor/viewer)	tenant owner
`documents`	Uploaded specs (PDF/DOCX/YAML)	tenant-scoped
`adapters`	8 pre-built adapters	global catalog
`adapter_versions`	Multi-version per adapter	FK → adapters
`configurations`	Generated integration configs	FK → adapter, doc
`config_history`	Versioned snapshots for rollback	FK → configs
`simulations`	Test run results	FK → configs
`simulation_steps`	Per-step test details	FK → simulations
`audit_logs`	Immutable action log	tenant-scoped
`webhooks`	Registered callback URLs	tenant-scoped
`webhook_deliveries`	Delivery attempts & status	FK → webhooks

UUID primary keys · Row-level tenant isolation · Alembic migrations · CASCADE deletes

Authentication & User Isolation

Security:

- JWT access (30min) + refresh (7d) tokens
- PBKDF2-SHA256 password hashing with salt
- Per-user data isolation (unique tenant ID)
- Role-based access: admin / editor / viewer

Each user sees only their own:

Documents, Configs, Simulations, Audit

Adapters are shared (global catalog).

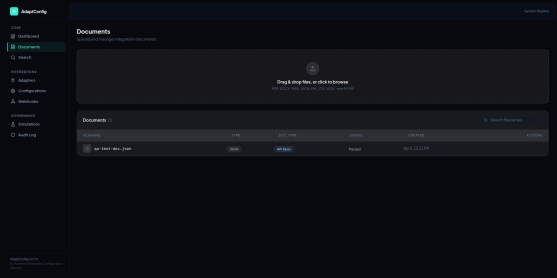
Additional protections:

- CORS restricted to known origins
- CSP, HSTS, X-Frame headers
- Rate limiting (100 req/min)
- AES-256 Fernet encryption
- Path traversal protection
- Alembic database migrations
- Structured JSON logging

Dashboard


AdaptConfig
System Healthy
CORE
Dashboard
Documents
Search
INTEGRATIONS
Adapters
Configurations
Webhooks
GOVERNANCE
Simulations
Audit Log
S Akash
Sign out
Configuration Summary
Total configuration
BY STATUS
Configured
1
AVG. CONFIDENCE
64%
across all mapped configurations
Weekly Activity
Documents & configurations processed
3
2.25
1.5
0.75
0
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Documents
Configs
Adapter Status
Current distribution
Active
1
Data Throughput
Records processed per hour
4 processed
2
1.5
1
0.5
0
00:00
04:00
08:00
12:00
16:00
20:00

Step 1: Upload & Parse



Formats: PDF, DOCX, YAML, JSON

Parser extracts:

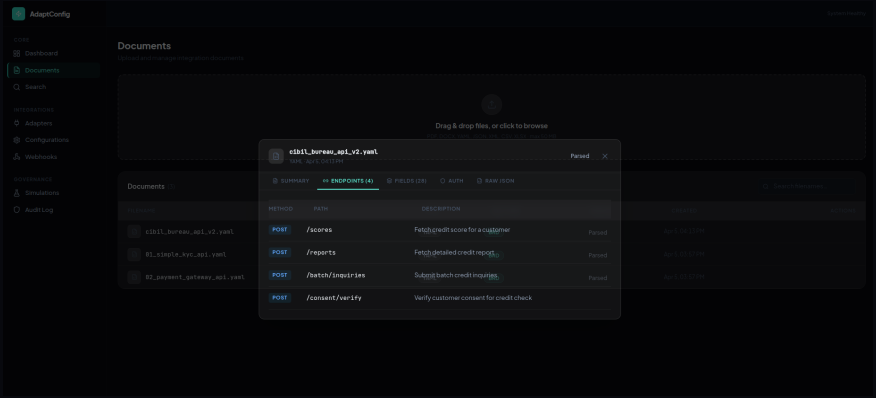
- API endpoints & methods
- Request/response fields
- Auth schemes
- SLA requirements

CIBIL v2 result:

4 endpoints · 28 fields · 2 auth

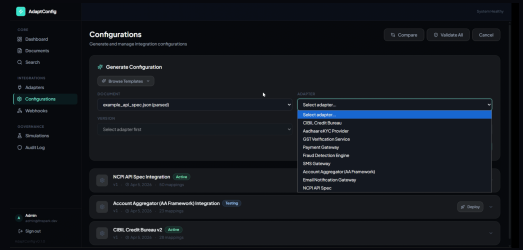
95% confidence

Parsed Document — 5-Tab Detail View



Summary · Endpoints · Fields (28) · Auth · Raw JSON

Step 2: AI Config Generation



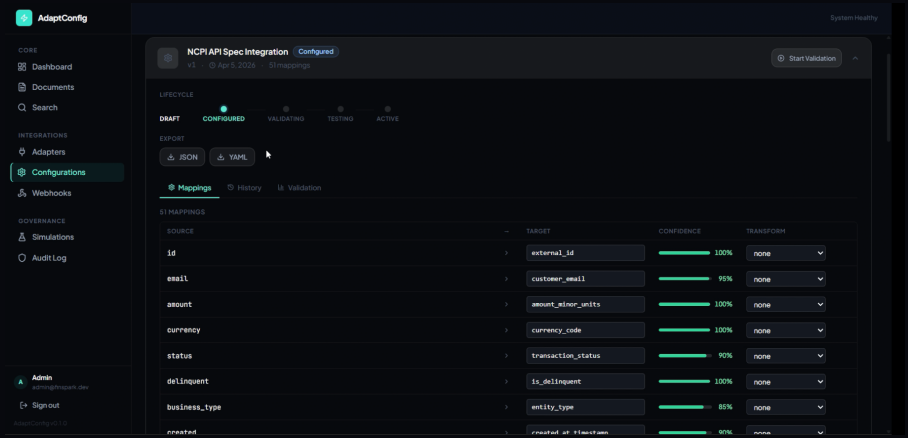
Two-stage pipeline:

1. GPT-4o generates config from document + adapter schema
2. **Rule engine** augments with fuzzy matching, Indian fintech synonyms, confidence scores

28/28 fields mapped at 100%

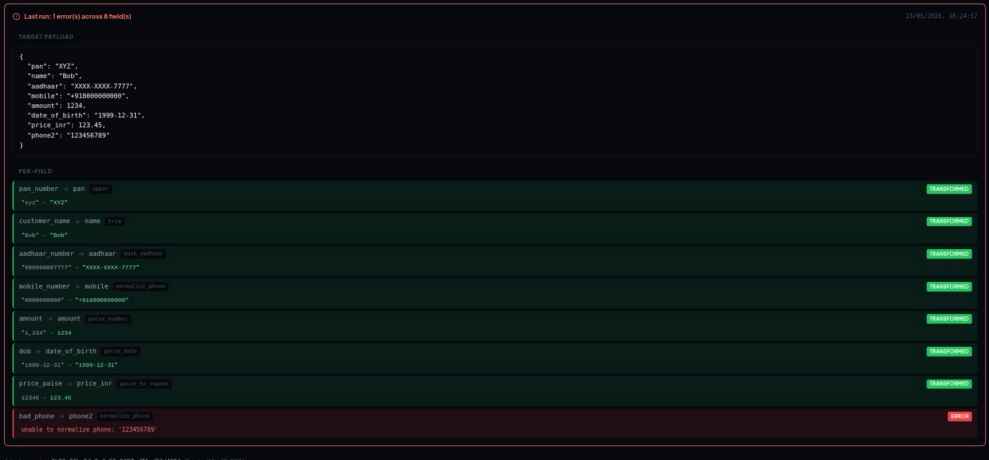
If no adapter matches, auto-detects and prompts to create one from the spec.

Editable Field Mappings



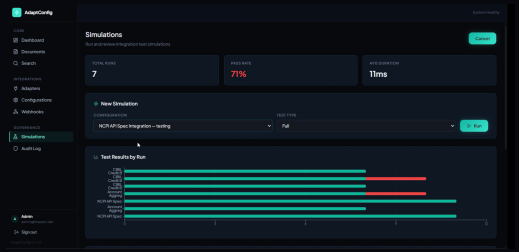
Lifecycle stepper · Editable targets · Transform dropdowns · Confidence bars · Export · History

Transformation Engine — Per-Field Results



Target payload preview · Per-field **TRANSFORMED** / **ERROR** status · One bad row never blocks the rest

Step 3: Simulation

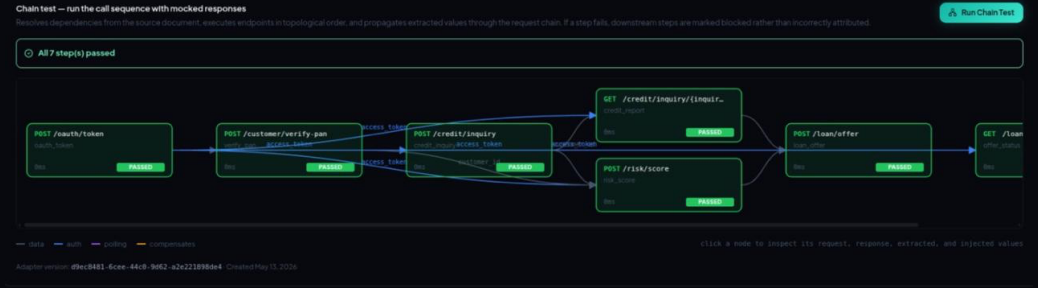


8-step pipeline:

- ✓ Config structure
- ✓ Field mapping coverage
- ✓ Endpoint: /scores
- ✓ Endpoint: /reports
- ✓ Endpoint: /batch/inquiries
- ✓ Endpoint: /consent/verify
- ✓ Auth validation
- ✓ Hooks validation

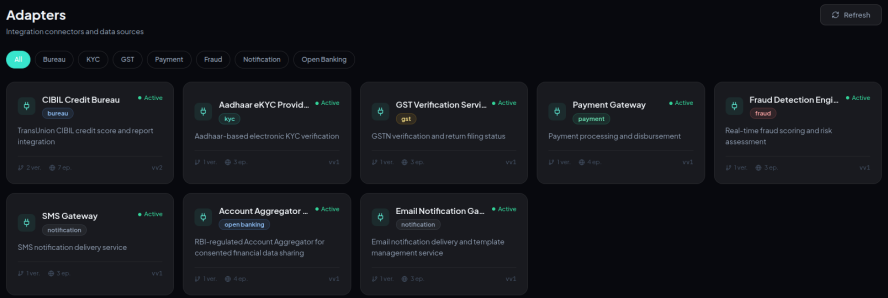
8/8 PASSED

Chain Test — Multi-Call API Linkage



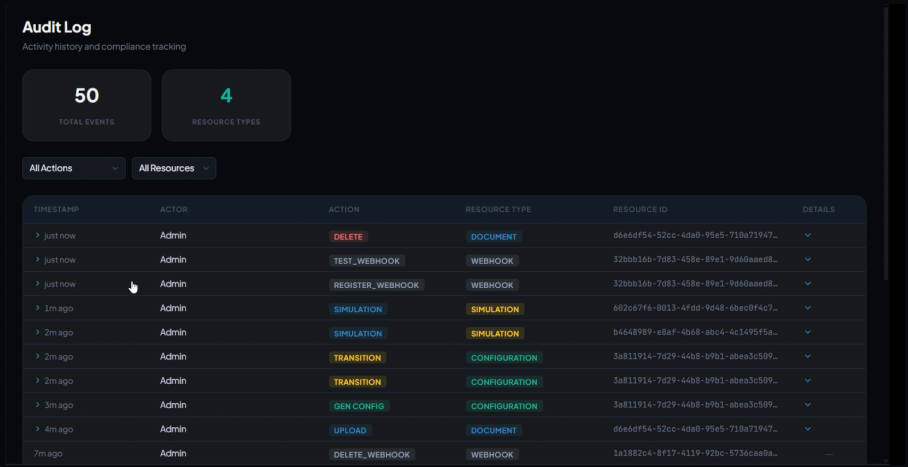
Topological execution from the source document · Extracted values propagate down the chain (`access_token`, `customer_id`, …) · Failed step → downstream marked **blocked**, never wrongly attributed

Adapter Catalog (8 Pre-Built + Custom)



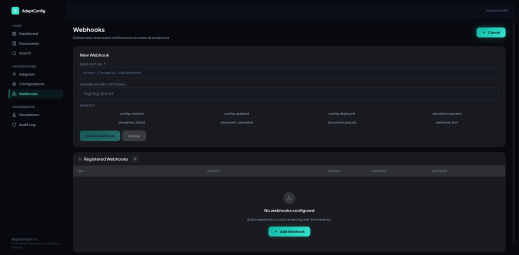
CIBIL · eKYC · GST · Payment · Fraud · SMS · Account Aggregator · Email · Custom

Audit Trail



Immutable log of every action · Filter by action type & resource · Pagination · Per-user isolation

Webhooks & Event System



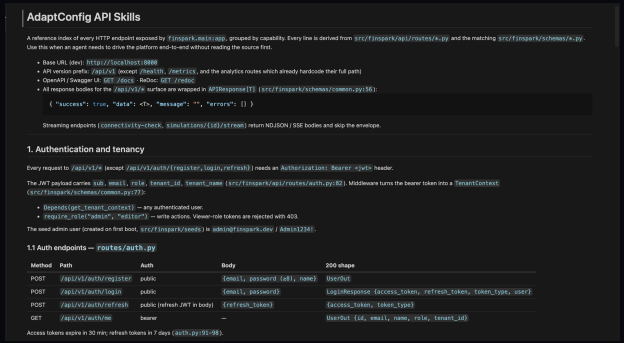
Event-driven notifications:

- Register webhook URLs for real-time callbacks
- Subscribe to events: config.created, simulation.completed, document.uploaded
- Test delivery with one click
- HMAC signature verification
- Delivery status tracking

Use cases:

- CI/CD pipeline triggers
- Slack/Teams notifications

Drop-in Skill: AdaptConfig as an Agent Tool



AdaptConfig API Skills

A reference index of every HTTP endpoint exposed by `finspark.main:app`, grouped by capability. Every line is derived from `src/finspark/api/routes/*.py` and the matching `src/finspark/schemas/*.py`. Use this when an agent needs to drive the platform end-to-end without reading the source first.

- Base URL (dev): `http://localhost:8000`
- API version prefix: `/api/v1` (except `/health`, `/metrics`, and the analytics routes which already hardcode their full path)
- OpenAPI / Swagger UI: `GET /docs` · ReDoc: `GET /redoc`
- All response bodies for the `/api/v1/*` surface are wrapped in `APIResponse[T]` (`src/finspark/schemas/common.py:56`):

```
{ "success": true, "data": <T>, "message": "", "errors": [] }
```

Streaming endpoints (`connectivity-check`, `simulations/{id}/stream`) return NDJSON / SSE bodies and skip the envelope.

1. Authentication and tenancy

Every request to `/api/v1/*` (except `/api/v1/auth/{register,login,refresh}`) needs an `Authorization: Bearer <jwt>` header.

The JWT payload carries `sub`, `email`, `role`, `tenant_id`, `tenant_name` (`src/finspark/api/routes/auth.py:82`). Middleware turns the bearer token into a `TenantContext` (`src/finspark/schemas/common.py:77`):

- `Depends(get_tenant_context)` — any authenticated user.
- `require_role("admin", "editor")` — write actions. Viewer-role tokens are rejected with 403.

The seed admin user (created on first boot, `src/finspark/seeds`) is `admin@finspark.dev` / `Admin1234!`.

1.1 Auth endpoints — `routes/auth.py`

Method	Path	Auth	Body	200 shape
POST	`/api/v1/auth/register`	public	`{email, password (≥8), name}`	`UserOut`
POST	`/api/v1/auth/login`	public	`{email, password}`	`LoginResponse {access_token, refresh_token, token_type, user}`
POST	`/api/v1/auth/refresh`	public (refresh JWT in body)	`{refresh_token}`	`{access_token, token_type}`
GET	`/api/v1/auth/me`	bearer	—	`UserOut {id, email, name, role, tenant_id}`

Access tokens expire in 30 min; refresh tokens in 7 days (`auth.py:91-98`).

One file. Any agent.

Ship a single `adaptconfig.skill.md` alongside the API. Claude / Cursor / any LLM picks it up and drives the platform end-to-end — mirrors the SPA 1:1:

upload → suggest adapter → generate → validate-and-test → deploy

Creativity — agents compose workflows we never coded.

Ease of use — no SDK, no SPA, just curl-able endpoints.

Results

	Manual	AdaptConfig	Improvement
Time per integration	2–4 weeks	$<$ 5 minutes	$\sim$500x faster
Field mapping errors	Common	AI-validated	Near zero
Pre-deploy testing	Rare	8-step pipeline	Always

Metric	Value
API Endpoints	34
Adapters	8
Tests	899
Coverage	82%
Pages	8

Performance	Value
Parse confidence	95%
Mapping accuracy	100%
Simulation	8/8 pass
Config gen	$\sim$ 30s
Doc parse	$\sim$ 10s

Live Demo

1. **Register / Login**
   Create account or use demo credentials
2. **Upload API Spec**
   Drag & drop YAML, PDF, or DOCX
3. **View Parsed Fields**
   28 fields, 4 endpoints, 2 auth schemes
4. **Generate Config**
   GPT-4o + Rule Engine pipeline
5. **Review Mappings**
   Edit targets, transforms, confidence bars
6. **Simulate & Test**
   8/8 steps pass with mock API responses
7. **Deploy**
   Configured → Validating → Testing → Active

Default admin: admin@finspark.dev / Admin1234!

Thank You

AdaptConfig

AI-Powered Integration Configuration Platform

Code `https://github.com/Akasxh/adaptconfig`

Team Nucleolus · S Akash · Swayam Jain · Yash Kamdar · IIT Patna